Python Programs Collection

All 10 uploaded programs compiled into one PDF

Includes the original Python code from files p1.py through p10.py.

Prepared from the provided files

Program 1: Multi-Day Bus Route Performance Analysis

Source code

```
routes = [
    ("Route A", 40, 50, 20, 10),
    ("Route B", 60, 50, 25, 12),
    ("Route C", 30, 40, 15, 8)
]

for name, passengers, capacity, time, fuel in routes:
    print("\nRoute:", name)
    print("Passengers:", passengers)

    if passengers > capacity:
        print("Status: Overloaded")
    else:
        print("Status: Normal")

    print("Fuel per passenger:", round(fuel / passengers, 2))
```

Program 2: Campus Event Seating and Entry Validation

Source code

```
seats = [[101, 102], [103, 104], [105, 106]]
registered = [101, 102, 103, 104, 105]
entered = [101, 102, 102, 105]

for ticket in entered:
    if entered.count(ticket) > 1:
        print(ticket, "Duplicate entry")
    elif ticket in registered:
        print(ticket, "Valid entry")
    else:
        print(ticket, "Invalid ticket")

print("Total Seats:", sum(len(row) for row in seats))
print("Available Seats:", sum(len(row) for row in seats) - len(set(entered)))
```

Program 3: Stock Inventory Reconciliation

Source code

```
received = [100, 150, 200]
distributed = [60, 100, 150]
damaged = [5, 10, 15]
remaining = [35, 40, 35]

for i in range(len(received)):
    expected = received[i] - distributed[i] - damaged[i]

    print("\nLocation", i + 1)
    print("Expected Stock:", expected)

    if expected != remaining[i]:
        print("Stock Discrepancy")
    else:
        print("Stock Correct")

    print("Replenishment Needed:", max(0, 100 - remaining[i]))
```

Program 4: Air Pollution Readings Analysis

Source code

```
readings = [
    [40, 50, 60],
    [70, 80, 90],
    [30, 45, 55]
]

for i, row in enumerate(readings):
    avg = sum(row) / len(row)

    print("Location", i + 1)
    print("Average:", avg)
    print("Maximum:", max(row))
    print("Minimum:", min(row))

    if avg > 60:
        print("Status: High Pollution")
    else:
        print("Status: Normal")
```

Program 5: Classroom Timetable Clash Detection

Source code

```
courses = [
    ("Python", "Room1", "9AM"),
    ("Maths", "Room2", "9AM"),
    ("English", "Room1", "10AM"),
    ("Web", "Room1", "9AM")
]

used = []

for course, room, period in courses:
    slot = (room, period)

    if slot in used:
        print(course, "Classroom Clash")
    else:
        print(course, "Allocated")
        used.append(slot)
```

Program 6: Bank Transaction Monitoring

Source code

```
balance = 1000
transactions = ["deposit 500", "withdraw 200",
                "withdraw 300", "withdraw 200"]

withdrawals = 0

for transaction in transactions:
    action, amount = transaction.split()
    amount = int(amount)

    if amount <= 0:
        print("Invalid amount")
    elif action == "deposit":
        balance += amount
        withdrawals = 0
    elif action == "withdraw":
        if amount <= balance:
            balance -= amount
            withdrawals += 1
            if withdrawals >= 3:
                print("Suspicious repeated withdrawals")
        else:
            print("Insufficient balance")

print("Final Balance:", balance)
```

Program 7: Exam Question Success Rate Analysis

Source code

```
key = "ABCDAB"
answers = ["ABCCAB", "BBCDAB", "ABDDAB"]

correct = [0] * len(key)

for answer in answers:
    for i in range(len(key)):
        if answer[i] == key[i]:
            correct[i] += 1

for i in range(len(key)):
    rate = correct[i] / len(answers) * 100
    print("Question", i + 1, "Success Rate:", rate, "%")

    if rate < 50:
        print("Difficult Question")
    else:
        print("Easy Question")
```

Program 8: Branch Stock Transfer Planning

Source code

```
stock = [100, 30, 80]
target = [60, 70, 80]

surplus = []
shortage = []

for i in range(len(stock)):
    difference = stock[i] - target[i]
    surplus.append(max(0, difference))
    shortage.append(max(0, -difference))

for i in range(len(stock)):
    for j in range(len(stock)):
        transfer = min(surplus[i], shortage[j])

        if i != j and transfer > 0:
            print("Transfer", transfer, "units: Branch",
                  i + 1, "to Branch", j + 1)
            surplus[i] -= transfer
            shortage[j] -= transfer
```

Program 9: Student Room Allocation

Source code

```
students = ["Ravi", "Sita", "Ram", "Priya", "Ravi"]
capacity = 2
rooms = [[], []]
assigned = set()

for student in students:
    if student in assigned:
        print(student, "Duplicate assignment")
        continue

    for room in rooms:
        if len(room) < capacity:
            room.append(student)
            assigned.add(student)
            break
    else:
        print(student, "No room available")

for i, room in enumerate(rooms):
    print("Room", i + 1, ":", room)

print("Unallocated Students:", len(students) - len(assigned))
```

Program 10: Building Electricity Consumption Analysis

Source code

```
buildings = {
    "Block A": [100, 120, 110],
    "Block B": [200, 220, 250],
    "Block C": [80, 90, 85]
}

cost_per_unit = 5
scores = []

for name, usage in buildings.items():
    total = sum(usage)
    average = total / len(usage)
    cost = total * cost_per_unit
    change = (usage[-1] - usage[0]) / usage[0] * 100

    print("\nBuilding:", name)
    print("Total Consumption:", total)
    print("Average:", average)
    print("Cost:", cost)
    print("Percentage Change:", round(change, 2), "%")

    if average > 150:
        print("Excessive Consumption")
    else:
        print("Normal Consumption")

    scores.append((name, total))

scores.sort(key=lambda x: x[1], reverse=True)
print("\nConsumption Ranking:")
for item in scores:
    print(item[0], item[1])
```